Task 8:

```
func average(of marks: Int...) -> Double {
    if marks.isEmpty {
        return 0
    }

    let total = marks.reduce(0, +)
    return Double(total) / Double(marks.count)
}
func grade(for average: Double) -> String {
    if average >= 90 {
        return "O"
    } else if average >= 80 {
        return "A+"
    } else if average >= 70 {
        return "A"
    } else if average >= 60 {
        return "B"
    } else {
        return "RA"
    }
}
func stats(of marks: [Int]) -> (min: Int, max: Int, average: Double)? {
    if marks.isEmpty {
        return nil
    }

    let minimum = marks.min()!
    let maximum = marks.max()!
    let avg = Double(marks.reduce(0, +)) / Double(marks.count)

    return (minimum, maximum, avg)
}
func curve(_ marks: inout [Int], by bonus: Int = 5) {
    for i in 0..<marks.count {
        marks[i] = min(marks[i] + bonus, 100)
    }
}
let avg = average(of: 85, 90, 75, 95)
print("Average:", avg)

print("Grade:", grade(for: avg))

let marks = [85, 90, 75, 95]
if let result = stats(of: marks) {
    print("Minimum:", result.min)
    print("Maximum:", result.max)
    print("Average:", result.average)
}

var studentMarks = [85, 90, 98, 70]
curve(&studentMarks)
print("Marks after bonus:", studentMarks)
```

Output

```
Average: 86.25
Grade: A+
Minimum: 75
Maximum: 95
Average: 86.25
Marks after bonus: [90, 95, 100, 75]
```

2. import Foundation

```
func isPalindrome(_ text: String) -> Bool {
    let cleaned = text
        .lowercased()
        .filter { $0.isLetter || $0.isNumber }

    return cleaned == String(cleaned.reversed())
}

func vowelCount(in text: String) -> Int {
    let vowels: Set<Character> = ["a", "e", "i", "o", "u"]

    return text.lowercased().filter { vowels.contains($0) }.count
}

func wordCount(in text: String) -> Int {
    return text.split { $0.isWhitespace || $0.isPunctuation }.count
}

func wordCount(in text: String, minLength: Int) -> Int {
    return text
        .split { $0.isWhitespace || $0.isPunctuation }
        .filter { $0.count >= minLength }
        .count
}

let text = "A man, a plan, a canal: Panama"
print("Text: \(text)")
print("Palindrome: \(isPalindrome(text))")
print("Vowel count: \(vowelCount(in: text))")
print("Word count: \(wordCount(in: text))")
print("Words with at least 4 letters: \(wordCount(in: text, minLength: 4))")
```

Output

```
Text: A man, a plan, a canal: Panama
Palindrome: true
Vowel count: 10
Word count: 7
Words with at least 4 letters: 3
```

3. import Foundation

```
func makeMultiplier(_ factor: Int) -> (Int) -> Int {
    return { number in
        return number * factor
    }
}

let double = makeMultiplier(2)
let triple = makeMultiplier(3)
func apply(_ values: [Int], using transform: (Int) -> Int) -> [Int] {
    return values.map { transform($0) }
}

let numbers = [1, 2, 3, 4, 5]

print("Original: \(numbers)")
print("Double: \(apply(numbers, using: double))")
print("Triple: \(apply(numbers, using: triple))")

let squares = apply(numbers, using: { number in
    return number * number
})

print("Squares: \(squares)")
```

Output

```
Original: [1, 2, 3, 4, 5]
Double: [2, 4, 6, 8, 10]
Triple: [3, 6, 9, 12, 15]
Squares: [1, 4, 9, 16, 25]
```